CCEE 324 – Computer Organization

Project Title:

Design and Simulation of a Simplified Dual-Core Processor for Arithmetic and Logical Operations

Project Objective:

In this project, you will design and simulate a simplified **two-core processor** capable of executing arithmetic and logical instructions.

Each processor core must independently fetch, decode, and execute instructions. Each core will have its own **Components**. The two cores will share a common **data memory**, allowing them to communicate by storing and retrieving data.

The project is intended to demonstrate how the basic components in Computer Organization can be combined to form a functioning processor and how the architecture can be extended from a single core to a dual-core system.

Project Instructions:

Design and implement a processor consisting of two independent CPU cores.

- Each core must contain a **program counter**, **register file**, **instruction decoder**, **control unit**, and **ALU**.
- The processor must support the following eight arithmetic and logical instructions:
  ADD, **SUB**, **AND**, **OR**, **XOR**, **NOR**, **SLL**, and **SRL**

  BONUS: You can expand to include additional instructions
- Each core must be capable of **independently** fetching, decoding, and executing instructions.
- The two cores will **share** a data memory, allowing one core to store a result that can subsequently be accessed by the other core.

Develop the processor using the **Verilog** or **VDHL** hardware description language or a **Python** processor simulator, or **Logisim** or **MARS**. Test all your individual components, and demonstrate independent and shared-memory execution.

Milestone:

Step 1: Design the single-core datapath

Step 2: Implement and test the ALU

Step 3: Duplicate the core and introduce independent execution

Step 4: Add shared memory and demonstrate dual-core operation

Deliverables:

1) Architecture/block diagram
2) Instruction-set specification
3) Instruction format
4) ALU design
5) Register-file design
6) Control-unit design
7) Single-core implementation
8) Dual-core implementation
9) Shared-memory implementation
10) Simulation results
11) Final technical report
12) Live demonstration

Submission Deadline:

Starting Wednesday, Nov. 25, 2026 by midnight. Presentations are required in person on campus.

Projects Hints

Program Counter:

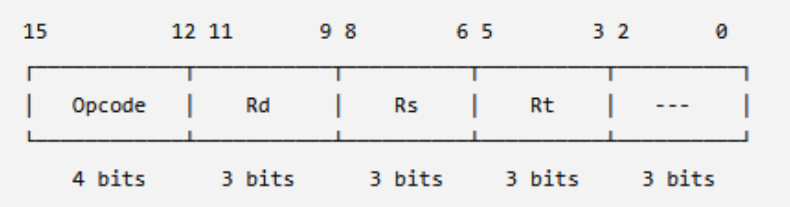
Each core must have its own PC. The PC must:

1. Identify the current instruction.
2. Advance to the next instruction after execution.
3. Stop when a HALT instruction is encountered.

Therefore, Core 0 PC ≠ Core 1 PC

Simplified Instructions:

Let's use a **16-bit register**.



Let's use an **8-bit processor** with **8 registers**.

Opcode	Instruction	Operation
0000	ADD	`Rd = Rs + Rt`
0001	SUB	`Rd = Rs - Rt`
0010	AND	`Rd = Rs & Rt`
0011	OR	`Rd = Rs | Rt`
0100	XOR	`Rd = Rs ^ Rt`
0101	NOR	`Rd = ~ (Rs | Rt)`
0110	SLL	`Rd = Rs << Rt`
0111	SRL	`Rd = Rs >> Rt`
1000	LOAD	`Rd = Memory[address]`
1001	STORE	`Memory[address] = Rs`
1111	HALT	`Stop execution`

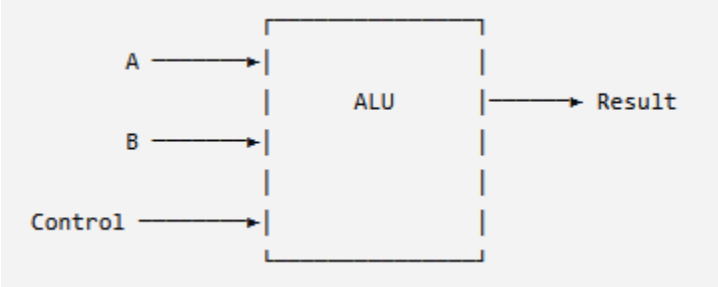
Register File:

Each core gets: 8 registers × 8 bits

Core 0 has its own: *R0 R1 R2 R3 R4 R5 R6 R7*

Core 1 has its own: *R0 R1 R2 R3 R4 R5 R6 R7*

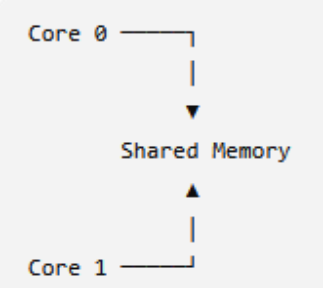
Arithmetic Logic Unit:



ADD SUB AND OR XOR NOR SLL SRL

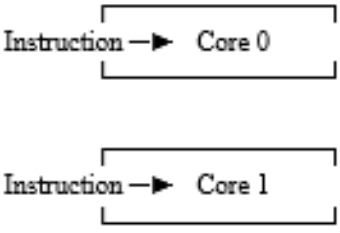
Shared Memory:

The two cores access the same memory.



Dual-core Operation:

For instance., although both cores could use R1, R2, and R3, they are **different physical registers**.



Processor Specifications:

Component	Requirement
Data width	`8 bits`
Number of cores	`2`
Registers / core	`8`
Register size	`8 bits`
Register names	`R0 – R7`
Data memory	`256 × 8 bits`
Instruction size	`16 bits`
Number of basic instructions	`11`
Execution	`Clock-based`